Railway level crossing in mCRL2

Diogo and Rodrigo Gomes de Araújo
up202510267, up202210267

November 2026

Introduction

This report was written for the first assignment of the *Systems Verification* course at *Faculdade de Ciências da Universidade do Porto* (FCUP). It presents a model of a railway level crossing, describing the interaction between a controller, a gate, and one or more trains that wish to cross. We then formalise a set of properties that verify its correctness and check them in mCRL2.

Task 1

In this task we modelled a single train passing through the crossing.

Components

The model contains the following three processes.

$$\begin{aligned} \text{Train} &= \text{approach} \cdot \text{grant} \cdot \text{enter} \cdot \text{exit} \cdot \text{leave} \cdot \text{Train} \\ \text{Gate} &= \text{lower} \cdot \text{down} \cdot \text{raise} \cdot \text{up} \cdot \text{Gate} \\ \text{Controller} &= \text{approach} \cdot \text{lower} \cdot \text{down} \cdot \text{grant} \cdot \text{leave} \cdot \text{raise} \cdot \text{up} \cdot \text{Controller} \end{aligned}$$

Each one consists of a loop over a finite set of actions. The `Controller` enforces the ordering between the `Gate` and the `Train`'s actions. It allows the train to enter (`grant`) after the gate is `down` and `raise` the gate after the train `leaves`.

Composition

Every interaction is modelled as a pair of complementary actions (e.g. `approach_train` and `approach_controller`) that `comm` merges into one synchronised action. `allow` then ensures they occur strictly simmultaneously.

```mcrl2
hide({ lower, grant, leave, raise },
allow({ approach, lower, down, grant, leave, raise, up, enter, exit },
comm({ approach_controller | approach_train -> approach, ... },
      TRAIN || GATE || CONTROLLER )));
```

For inspection, we split the actions into messages and physical events. Messages are internal handshakes between the components and cannot be observed from the outside, so we hide them:

- **`lower`, `raise`:** the commands to the gate;
- **`grant`:** the permission to cross;
- **`leave`:** the train's notification that it left.

Physical events, on the other hand, are what someone standing near the crossing would observe, so they remain visible:

- **`approach`:** the train arriving;
- **`enter`, `exit`:** being on the crossing;
- **`down`, `up`:** the gate reaching its position.

Inspection

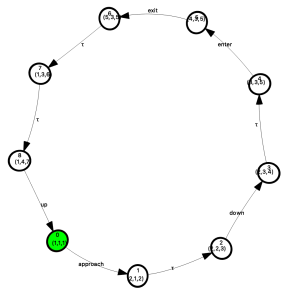


Figure 1: Full LTS

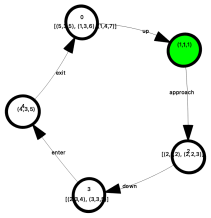


Figure 2: Reduced LTS

The full LTS is a single cycle of 9 states, four of whose transitions are τ (the hidden messages). None of these τ-steps removes a choice, so branching bisimulation collapses them, leaving a cycle of 5 states made only of the physical events:

$$\text{approach} \to \text{down} \to \text{enter} \to \text{exit} \to \text{up} \to \text{approach}$$

This is the externally observable behaviour of the train passing through the crossing.

Properties

Property	Formula	Result
Safety	$[-^*;\text{enter};(-\text{exit})^*;\text{up}]\ \text{false} \wedge [(-\text{down})^*;\text{enter}]\ \text{false} \wedge$ $[-^*;\text{up};(-\text{down})^*;\text{enter}]\ \text{false}$	true
Reachability	$[-^*;\text{leave}]\langle -^*;\text{raise}\rangle\ \text{true}$	true
Liveness	$[-^*;\text{approach}]\langle -^*;\text{exit}\rangle\ \text{true}$	true
Deadlock freedom	$[-^*]\langle -^*;\text{approach}\rangle\ \text{true}$	true

The model satisfies four properties, and all of them hold. For *safety*, the train is never on the crossing while the gate is open: once a train has entered (`enter`), the gate cannot go up (`up`) until the train exits (`exit`), and a train can only enter if the gate has gone down (`down`) since it was last raised, which together guarantee the gate is closed whenever the train is on the crossing. For *reachability*, from any reachable state the gate can eventually be raised (`raise`), so the system can always return to its open state. For *liveness*, whenever a train approaches, it can eventually exit, so an approaching train is never blocked forever. Finally, for *deadlock freedom*, from any reachable state a new train can eventually approach (`approach`), so the system never gets stuck and the cycle can always restart.

Broken Variant

In the faulty variant, the controller issues the grant immediately after ordering the gate to lower, without waiting for the `down` acknowledgement:

```
1  1      approach_controller                                    mcrl2
2     -   . lower_controller . down_controller
3     -   . grant_controller
   2  +   . lower_controller . grant_controller
   3  +   . down_controller
```

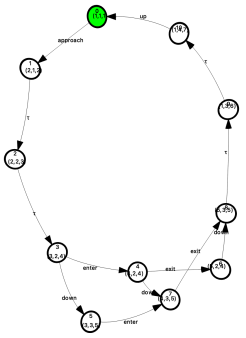


Figure 3: Broken LTS

The safety property $[(\neg\ \text{down})^* \cdot \text{enter}]\ \text{false}$ (no `enter` unless a `down` occurred since the last `up`) fails with the following counterexample:

approach . lower . grant . enter

Since `lower` and `grant` are hidden, the observable trace is just `approach, enter`: the train enters with no `down` before it, so the formula is violated. The gate process is still waiting to fire `down_gate`, because after `lower` the train and gate run independently and the train wins the race.

Task 2

In this task the model allows multiple trains to pass through the crossing one at a time.

Model with data

$\text{Train}(t)$ and Gate are the same as in Task 1, with the train's actions carrying its identity. The controller keeps a queue Q of the trains that approached and did not leave yet, whether the crossing c is `busy` or `free`, and the gate state g:

$$\begin{aligned}\text{Controller}(Q, c, g) = \sum_{t \in \text{Id}} (t \notin Q) \to \text{approach}(t)\ .\ \text{Controller}(Q \triangleleft t, c, g)\\ +(Q \neq [\,] \wedge g = \text{closed} \wedge c = \text{free}) \to \text{grant}(\text{head}(Q))\ .\ \text{Controller}(Q, \text{busy}, g)\\ +(Q \neq [\,] \wedge c = \text{busy}) \to \text{leave}(\text{head}(Q))\ .\ \text{Controller}(\text{tail}(Q), \text{free}, g)\\ +(g = \text{open} \wedge Q \neq [\,]) \to \text{lower}\ .\ \text{Controller}(Q, c, \text{closing})\\ +(g = \text{closing}) \to \text{down}\ .\ \text{Controller}(Q, c, \text{closed})\\ +(g = \text{closed} \wedge Q = [\,] \wedge c = \text{free}) \to \text{raise}\ .\ \text{Controller}(Q, c, \text{opening})\\ +(g = \text{opening}) \to \text{up}\ .\ \text{Controller}(Q, c, \text{open})\end{aligned}$$

where $Q \triangleleft t$ appends t to the end of Q. We chose a queue over a set so that only the first train in line can be granted. No train is overtaken, which matters for liveness.

Invariant. Q always holds exactly the trains that approached but did not leave yet: `approach(t)` is the only step that adds a train and `leave` the only one that removes it. c is `busy` exactly while one train is between `grant` and `leave`. Since `grant` needs $c = \text{free}$, only one train crosses at a time, and since `raise` needs Q empty and $c = \text{free}$, the gate is never raised while a train is near.

Properties

All properties are quantified over $t \in \text{Id}$, and over $t' \neq t$ where t' appears.

Property	Formula	Result
1. Safety	$[-^*; \text{enter}(t); (-\text{leave}(t))^*; \text{up}]\ \text{false}$	true

2. Mutual exclusion	$[-^*; \mathrm{enter}(t); (-\mathrm{exit}(t))^*; \mathrm{enter}(t')]$ false	true
3. No premature raise	$[-^*; \mathrm{approach}(t); (-\mathrm{leave}(t))^*; \mathrm{raise}]$ false	true
4. Deadlock freedom	$[-^*]\langle -^*; \mathrm{approach}(t)\rangle$ true	true
5a. Liveness (reach)	$[-^*; \mathrm{approach}(t); (-\mathrm{enter}(t))^*]\langle -^*; \mathrm{enter}(t)\rangle$ true	true
5b. Liveness (order)	$[-^*; \mathrm{approach}(t); (-\mathrm{enter}(t))^*; \mathrm{enter}(t');$ $(-\mathrm{enter}(t))^*; \mathrm{enter}(t')]$ false	true
6. Road availability	$[-^*]\langle -^*; \mathrm{up}\rangle$ true	true
7. Infinite opening	$[((-\mathrm{up})^*; \mathrm{up})^*]$ true $\wedge \langle -\rangle$ true	true
8. Non-vacuity	$[-^*]\langle -^*; \mathrm{enter}(t)\rangle$ true $\wedge [-^*]\langle -^*; \mathrm{raise}\rangle$ true $\wedge [-^*]\langle -\rangle$ true	true

Property 3 checks `raise`, the controller's decision to open. Once raised, the gate always finishes with `up`, even if a train approached meanwhile; the controller then lowers it again before any `grant`. Property 8 makes sure the premises of 1–3 are reachable, so they do not hold vacuously.

Liveness and fairness (5). Without fixpoints we cannot say that `enter(t)` **must** happen, so we check two things: `enter(t)` stays reachable while t waits (5a), and no other train enters twice in the meantime (5b) ($t' \neq t$, since t's own next trip is normal). With a set, a train could be overtaken forever and we would need to assume fairness between trains. The queue removes that: a waiting train is overtaken at most $N - 1$ times, so we only assume the system keeps running (property 4).

Infinite opening (7). The controller cannot guarantee this. If trains keep arriving, Q never empties and the gate never opens. A counterexample would mean that, under heavy traffic, cars wait for as long as trains keep coming. This is a loss of availability, not of safety, and property 6 shows that opening is always still possible.

Broken Variant

Conclusion